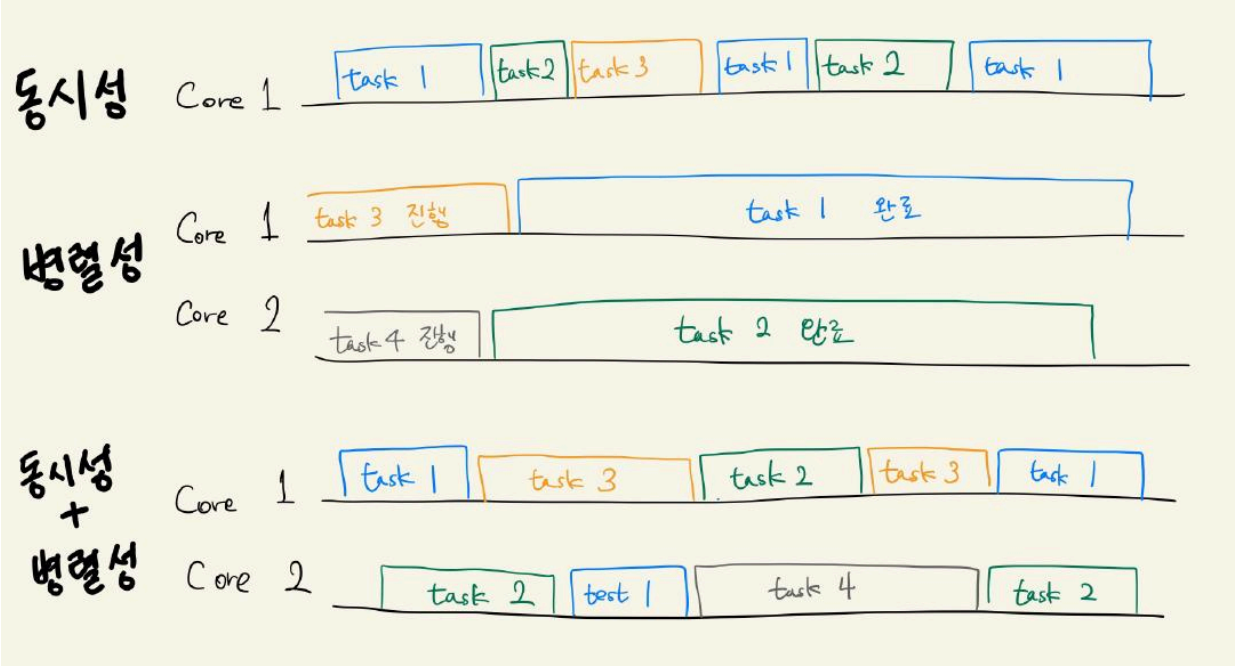
동시성(Concurrency)과 병렬성(Parallelism)에 대하여

동기와 비동기 방식, 동시성(Concurrency)과 병렬성(Parallelism)을 구현하는 세 가지 핵심 라이브러리(threading, multiprocessing, asyncio)의 차이점과 작동 방식으로 관련 코드는 "어떤 상황에서 어떤 도구를 써야 하는가?"에 대한 답을 코드로 증명하는 코드로 관련 코드는

- ☐ 동시성 : 0422_sync_async_clean.ipynb : 동기 비동기 방식 차이 설명
- ☐ 동시성과 병렬성 : 0422_concurrency.py : 작업의 성격(계산이 힘든가, 기다리는 게 일인가)에 따라 파이썬이 일을 시키는 세 가지 방법을 아주 명확하게 대조해서 보여주는 가이드북 코드



[Concurrency vs Parallelism]

---

1. Threading (스레드): I/O Bound 작업용

run_with_threads 함수는 여러 개의 스레드를 만들어 작업을 처리합니다.

- 핵심 원리**:** 파이썬에는 GIL(Global Interpreter Lock)*이 있어 한 번에 하나의 스레드**만 파이썬 바이트코드를 실행할 수 있지만 네트워크 응답을 기다리거나 파일을 읽는 **I/O** 작업 중에는 GIL이 일시적으로 해제
  - GIL(Global Interpreter Lock)* : 파이썬 인터프리터(주로 표준인 CPython)가 한 번에 오직 하나의 스레드만 파이썬 바이트코드를 실행할 수 있도록 제어하는 잠금 장치
  - 스레드(Thread)**는 프로세스 내에서 실행되는 '최소 작업 단위'
- 결과**:** 3초 걸리는 작업과 2초 걸리는 작업을 동시에 시작하여, 총 약 **3**초 만에 종료. (순차 실행 시 5초 소요)

2. Multiprocessing (멀티프로세스): CPU Bound 작업용

run_with_process_pool 함수는 ProcessPoolExecutor를 사용하여 여러 개의 CPU 코어를 활용합니다.

- 핵심 원리: 소수 판별 같은 복잡한 연산은 CPU를 계속 사용하므로 GIL 때문에 스레드로는 속도 향상을 볼 수 없지만 대신 완전히 독립된 프로세스를 여러 개 띄워 각 코어에 할당함으로써 GIL의 제약을 물리적으로 뛰어넘음
- 결과: 여러 개의 CPU 코어를 동시에 사용하여 연산 속도를 비약적으로 높임

3. Asyncio (코루틴): 비동기 I/O 작업용

run_with_coroutines 함수는 단일 스레드 내에서 await 키워드를 사용해 제어권을 주고받음

- 핵심 원리: 스레드를 여러 개 만드는 '비싼' 비용을 들이지 않고, 하나의 스레드 안에서 "A가 쉴 때 B를 한다"는 개념으로. await 문자를 만나면 이벤트 루프가 다른 작업으로 고개를 돌려 일을 처리
- 결과: 스레드 방식과 비슷하게 약 3초가 걸리지만, 메모리 리소스를 훨씬 적게 사용하므로 수천 개의 연결을 처리할 때 매우 유리

요약 비교표

방식	모듈	주 용도	핵심 키워드	특징
Multi-threading	threading	I/O (웹 서핑, 파일 읽기)	Thread, join	GIL 영향 받음, 공유 메모리 사용 편함
Multi-processing	multiprocessing	CPU (연산, 이미지 처리)	ProcessPool, map	GIL 우회, 리소스 소모 큼, 병렬 처리
Asyncio	asyncio	대규모 I/O (채팅 서버 등)	async, await, gather	단일 스레드, 협력적 멀티태스킹